

Caching Responses of Other Services

1. Overview

In modern distributed systems, one service often depends on **external services** or **downstream internal microservices** for data such as:

- Payment gateway response
- User profile service
- Recommendation engine
- Currency exchange rates
- Product inventory
- Maps / geolocation API
- Email / SMS provider
- Analytics provider

If one of these services becomes **slow, unavailable, or returns errors**, your system can also fail unless designed properly.

This is where **cached fallback responses** help improve **fault tolerance**.

2. Core Idea

Instead of directly failing when dependency is down:

- Store successful responses from dependent services in cache
- If service becomes unavailable later:
 - Return cached data
 - Return stale data
 - Return default response
 - Return partial response

This approach enables **graceful degradation**.

3. Why This Matters

Without caching:

None

User Request → Our Service → External Service Down → Entire Request Fails

With caching:

None

User Request → Our Service → External Service Down



Return Cached Response

System remains functional.

4. Graceful Degradation

Graceful degradation means:

Instead of full outage, system gives reduced but usable functionality.

Examples:

Example 1: E-commerce

If recommendation service fails:

- Show cached recommendations
- Show popular products
- Hide recommendation section

Example 2: Weather App

If weather API down:

- Show last known forecast
- Show “Updated 20 mins ago”

Example 3: Payment Fraud Check Down

- Queue transaction for later verification
- Allow low-risk payments temporarily

5. How Cached Response Strategy Works

Step 1: Successful Call

None

Request User Profile



Profile Service returns data



Store response in Redis / Memory Cache

Step 2: Dependency Failure

None

Profile Service Timeout / Error



Read last cached profile



Return cached response

6. Cache Types for Fault Tolerance

A. In-Memory Cache

- Fastest
- Stored inside service memory
- Good for single instance

Examples:

- LRU cache
- HashMap cache

B. Distributed Cache

Used in production systems.

Examples:

- Redis
- Memcached

Benefits:

- Shared across instances
- Survives service restarts
- Centralized cache

C. CDN / Edge Cache

Useful when external data is static.

Examples:

- Images
- Public metadata

7. Stale Data vs No Data

Often in distributed systems:

None

Stale data > No data

Examples:

Use Case	Accept Stale?
Stock price	No
Product catalog	Yes
User profile name	Yes
Bank balance	No
Weather	Sometimes
Currency rates	Limited

8. TTL (Time To Live)

Every cache entry should expire.

Example:

None

Exchange Rate Cache TTL = 5 min

User Profile TTL = 30 min

Recommendations TTL = 1 hour

Prevents indefinite stale data.

9. Cache Fallback Flow

None

Try Live API

|

Success? ---- Yes → Return + Update Cache

|

No

↓

Cached Value Exists?

|

Yes → Return Cached Value

No → Return Default / Error

10. Real World Example

Ride Booking App

Depends on Maps API.

If Maps API fails:

Instead of breaking:

- Use cached route estimates
- Use approximate fare
- Show delayed ETA

Users still book rides.

11. Important Design Considerations

A. Data Freshness

Need to know:

- How stale is acceptable?
- When to reject old data?

Example:

None

News headlines stale after 5 mins

User profile okay after 1 day

B. Cache Invalidation

Hardest problem in caching.

Need to remove/update when source changes.

Techniques:

- TTL expiry
- Write-through cache
- Event-driven invalidation

C. Versioning

Store schema version with cache.

Useful when response format changes.

D. Security

Never cache sensitive data carelessly:

- Passwords
- Tokens
- Payment details
- Personal health data

Encrypt if needed.

12. Combine with Circuit Breaker

Best practice:

When service repeatedly fails:

None

Circuit Breaker Opens



Stop calling failing dependency



Use cache fallback directly

Prevents repeated timeouts.

13. Interview Answer

Q: How do you handle dependency failure in HLD?

Answer:

I use graceful degradation. Successful dependency responses are cached in Redis or memory. If dependency fails, I return cached or stale responses depending on business criticality. I combine this with timeouts, retries, circuit breaker, and monitoring.

14. Best Practices

- Set proper TTL
- Define fallback policy per API
- Use Redis for distributed systems
- Monitor cache hit ratio
- Mark stale responses if needed
- Combine with retry + circuit breaker
- Never cache sensitive data blindly

15. Short Revision Notes

None

Dependency Down?

→ Don't fail immediately

Use:

1. Cached Response
2. Stale Response
3. Default Response
4. Partial Response

Goal:

Graceful degradation

Tools:

Redis, Memcached, In-memory cache

Combine with:

Timeout + Retry + Circuit Breaker